

Complexity-Driven Pruning for Implicit Constructive Solid Geometry

Egor Chulkov
Oleg V. Vasilyev

Center for Materials Technologies, Skolkovo Institute of Science and Technology, Moscow, Russia
Egor.Chulkov@skoltech.ru, O.Vasilyev@skoltech.ru

Preprint

Abstract

A complexity-driven acceleration method for implicit constructive solid geometry (CSG) models composed of closed-form primitives and set-theoretic operations is presented. Unlike existing sparse hierarchies that regulate memory via spatial criteria despite runtime being governed by expression complexity, in the proposed method spatial subdivision is coupled directly to pruned local expression complexity. Guaranteed bounds from interval arithmetic are leveraged to prune the global CSG expression per voxel, with refinement terminated when residual complexity falls below a user-defined complexity threshold. The resulting hierarchies adapt to both geometric detail and the algebraic structure of CSG expressions, effectively decoupling evaluation cost from spatial resolution. Compared to conventional max-depth approaches, sparser hierarchies with equivalent local complexity are generated, reducing traversal overhead and memory footprint while ensuring robust near-surface evaluation. Experiments across diverse detailed constructive scenes demonstrate more predictable scaling behavior and improved total speedup.

Keywords: implicit CSG, complexity-driven pruning, adaptive subdivision, sparse octrees, function representation

1 Introduction

Since the formal introduction of constructive solid geometry (CSG) by Ricci in 1973, which established a theoretical basis for solid modeling, implicit (or functional) representation has received relatively little attention as a unified geometric modeling paradigm. Early developments focused on establishing the mathematical formalism of R-functions Rvachev (1975) and exploring basic set-theoretic operations for implicit geometric primitives Pasko et al. (1995). Until recently, the general field of geometric modeling was dominated by explicit boundary representations Requicha (1980) due to their intuitive control and efficient visualization capabilities. Consequently, implicit CSG was typically utilized in specialized settings, including robust solid modeling and classification via interval-based subdivision Duff (1992), voxel-based and sparse volumetric processing Frisken et al. (2000); Museth (2013), and visualization / interactive systems that evaluate closed-form implicit expressions efficiently Keeter (2020); Uchytil and Storti (2023). More recently, interest has been renewed by digital fabrication and design workflows that benefit from exact Boolean operations, interior-aware representations, and scalable field interrogation Maltsev et al. (2021); Uchytil and Storti (2023); Popov et al. (2021).

Implicit modeling represents a solid object via a scalar field $F(\mathbf{x})$, where the boundary of the object is given by an isosurface, typically $F(\mathbf{x}) = 0$. Compared to boundary representation (B-rep), implicit

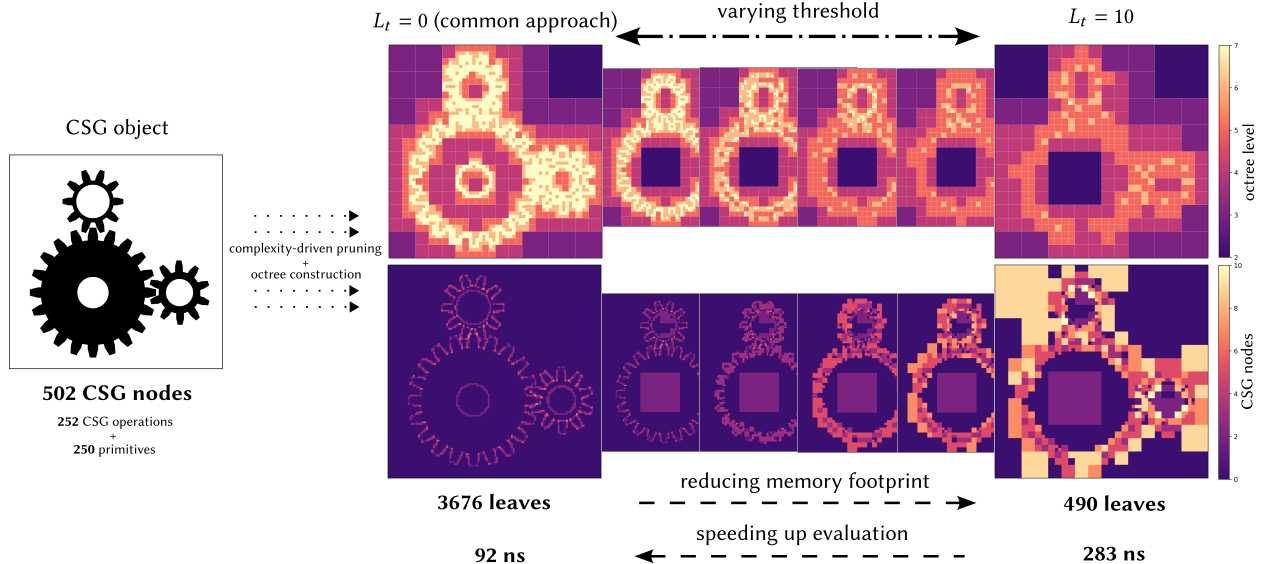


Figure 1. The complexity-performance trade-off in implicit CSG. Increasing the complexity threshold L_t terminates boundary refinement earlier, yielding sparser hierarchies with reduced traversal overhead (fewer leaves), but retaining more complex pruned local expressions per leaf. Decreasing L_t forces finer subdivision to simplify local expressions, eventually approaching the conventional spatial max-depth baseline at $L_t = 0$.

representations offer attractive properties for highly detailed shapes and constructive editing: Boolean operations are naturally expressed as function compositions, and operations such as blending, offsets, and procedural detailing can be defined directly on the field Pasko et al. (2002, 2011). This family includes function representation (F-rep) Pasko et al. (1995), Signed Distance Fields (SDFs), and hierarchical implicit systems such as BlobTrees Wyvill et al. (1999). In F-rep, CSG operations are commonly encoded through R-functions Rvachev (1975), while SDF-based methods often use min/max compositions for hard CSG and smooth variants for blending.

Rendering and interrogation of implicitly defined objects are typically performed by either polygonization or ray-based methods. Polygonization approaches, e.g., Marching Cubes Lorensen and Cline (1987) and Dual Contouring Ju et al. (2002), are often the fastest route to interactive visualization, but introduce discretization error and require regeneration when the underlying model changes. Ray-based methods evaluate the implicit function along view rays and are attractive when exact surface visualization and scalability with respect to resolution are important. For SDFs, the most widely used approach is sphere tracing, which uses distance-based steps to advance safely toward the first intersection Hart (1996). For general F-rep, intersection is commonly reduced to root finding, e.g., bracketing plus bisection, which can be made robust using range tests based on interval-style enclosures Snyder (1992); Duff (1992); Knoll et al. (2009).

A major practical bottleneck is the cost of repeated function evaluation: ray-casting and root finding may require thousands of evaluations per pixel. Similarly, in geometric modeling, point classification often dominates the runtime for complex CSG trees. To accelerate these computations, two complementary approaches have been pursued. The first uses spatial hierarchies that store near-surface information to avoid evaluating the full expression in empty space, for instance, through adaptive sampling Frisken et al. (2000) or sparse volumetric representations in modern kernels Keeter (2020); Uchytel and Storti (2023). The second is expression pruning, which removes operations that cannot affect the result in a local region, resulting in locally simplified functions that are cheaper to evaluate. For closed-form implicits, GPU-oriented interpreters with pruning of min/max-heavy expressions have demonstrated interactive performance Keeter (2020), and sparse volumetric CAD kernels have combined adaptive subdivision with local, pruned function variants

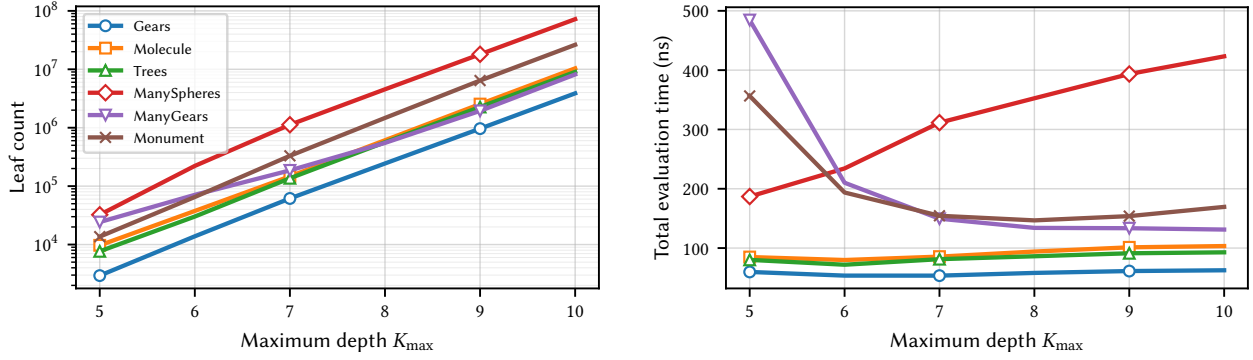


Figure 2. Motivation for complexity-driven termination: increasing maximum depth can reduce per-voxel evaluation cost due to smaller pruned expressions, but increases traversal overhead and memory footprint, resulting in a scene-dependent optimum.

Uchytíl and Storti (2023). For primitive-based SDFs, hierarchical simplification based on conservative bounds, e.g., Lipschitz pruning, can also prune aggressively while remaining compatible with sphere tracing Barbier et al. (2025).

Despite these advances, existing sparse-hierarchy pipelines typically control memory by enforcing a fixed maximum resolution, such as an octree maximum depth. This parameter requires careful tuning: deeper trees reduce the average cost of evaluating the *pruned* local function, but increase hierarchy traversal overhead and memory footprint. In practice, the optimal choice is scene-dependent, and increasing maximum depth may either improve or degrade end-to-end performance depending on the distribution of local expression complexity. This makes the outcome of pruning less predictable and complicates parameter selection for models with widely varying complexity. Figure 2 summarizes the practical consequences of relying on a fixed maximum depth: the memory footprint of sparse volume hierarchies can grow rapidly with increased resolution, while total evaluation time often exhibits a scene-dependent optimum due to the trade-off between hierarchy traversal cost and the cost of evaluating the (pruned) local expression.

This paper introduces a complexity-driven construction paradigm that fundamentally couples the topology of the hierarchy to the local algebraic structure of the implicit model. Unlike fixed-depth or geometric criteria ($K_{\max}$), which remain agnostic to the underlying evaluation cost of the CSG expression, the proposed approach employs a termination rule that forces spatial subdivision to adapt explicitly to local evaluation complexity. Intuitively, the topology of the hierarchy is no longer an input parameter, but an emergent property derived from the distribution of complexity inherent to the implicit CSG model.

Figure 1 previews the resulting behavior. Increasing the threshold L_t terminates boundary voxels earlier, substantially reducing octree occupancy (and thus memory and traversal cost), but retaining larger voxel-local expressions that increase per-query evaluation cost. Decreasing L_t has the opposite effect, approaching the conventional max-depth behavior which is recovered as the special case $L_t = 0$. Overall, the proposed threshold-based strategy provides direct control over the memory–performance trade-off, leading to more predictable acceleration and substantially reduced memory footprints for a given runtime target in highly detailed scenes.

Contributions. The contributions of this work are summarized as follows:

- A *threshold-based stopping criterion* is introduced for adaptive spatial subdivision, terminating refinement based on the *local evaluation complexity* of the pruned implicit model. The proposed strategy provides direct control over the memory–time trade-off via a single complexity parameter, with the standard max-depth strategy recovered as a special case.

- A pruning algorithm is proposed for implicit CSG model that constructs locally simplified expressions optimized for *near-surface* tasks such as ray-casting and contouring, enabling faster evaluation by skipping operations provably irrelevant to the result within each voxel.
- A systematic evaluation of the proposed threshold-based criterion against the conventional *max-depth* termination rule is presented. The method is benchmarked against representative pruning baselines using metrics for pruned-function complexity, traversal overhead, memory footprint, and runtime.

Scope. This work focuses on spatial acceleration for constructive implicit modeling in computer graphics and computer-aided design (CAD). The paper studies implicit solids built from closed-form primitives combined by hard CSG operators, targeting workloads dominated by repeated field queries. The scope is limited to algorithmic comparisons between pruning strategies and termination criteria. Implementation-level optimizations, such as GPU acceleration, are not considered, and thus performance results are interpreted as relative measures between the evaluated methods.

Paper organization. A review of related work on acceleration of evaluation of implicit functions through spatial hierarchies and expression pruning is provided in Section 2. A novel threshold-based stopping criterion is proposed in Section 3, followed by a description of the pruning algorithm in Section 4. The rendering and evaluation pipeline is outlined in Section 5. Section 6 presents the experimental results and comparisons, while conclusions, limitations, and future work are discussed in Section 7.

2 Related Work

Spatial hierarchies and acceleration Spatial acceleration is fundamental for efficient implicit modeling. Adaptively Sampled Distance Fields (ADFs) store sampled data in octrees refined near the surface Frisken et al. (2000), while Sparse Voxel Octrees (SVOs) enable efficient ray traversal via empty-space skipping Laine and Karras (2010); Amanatides and Woo (1987). In production environments, VDB Museth (2013) and its GPU-optimized variants, such as NanoVDB Museth (2021) and GVDB Hoetzlein (2016), provide robust sparse topological management. For rendering, traversal is often accelerated by specialized intersection algorithms like sphere tracing for SDFs Hart (1996) or synchronized ray streaming Zanni (2024). However, in most existing systems, the construction of the acceleration structure is driven primarily by *geometric* criteria, such as surface proximity or a fixed maximum resolution, rather than the *computational cost* of the underlying field evaluation Uchytel and Storti (2023).

Interval analysis and expression pruning Interval arithmetic (IA) provides the rigorous inclusion guarantees necessary for robust root finding and classification Snyder (1992). Duff (1992) pioneered the use of IA for spatial subdivision, deriving *sign-preserving* pruned CSG trees to cull sub-expressions that do not affect the surface classification. While effective for basic inclusion tests, standard IA suffers from overestimation (the dependency problem), motivating the use of affine arithmetic Knoll et al. (2009); Fryazinov et al. (2010) to tighten bounds. More recently, Keeter (2020) adapted pruning for *value-preserving* tasks, simplifying min/max expressions for interactive rendering while retaining the exact field value. Similarly, Barbier et al. (2025) utilized Lipschitz bounds to prune primitive-based SDFs during sphere tracing. However, these methods typically apply pruning *locally* within a hierarchy fixed by spatial constraints. In contrast, the proposed approach allows the efficiency of the pruned expression itself to determine the topology of the hierarchy.

Termination criteria in adaptive subdivision Most hierarchical approaches require a stopping criterion to balance memory and precision. Common choices include a fixed maximum depth (standard in SVOs/VDB), geometric error thresholds (ADFs), or stopping when range tests become decisive (interval subdivision). However, fixed-depth criteria can be ill-suited for constructive models with non-uniform complexity: deep refinement is wasteful in computationally simple regions, yet potentially insufficient in complex ones. While heuristics such as “one primitive per voxel” have been proposed Dyllong and Grimm (2008), they lack a formalized mechanism to balance memory against runtime. In contrast, the proposed method formalizes *local complexity* as a tunable constraint. This decouples subdivision from fixed geometric targets, providing predictable guarantees on the residual evaluation cost.

3 Threshold-based stopping criterion

Standard construction algorithms for spatially accelerated implicit models typically rely on purely spatial termination criteria. Hierarchies such as octrees or N^3 -trees are subdivided until a voxel is either proven empty or a predetermined maximum resolution (maximum depth) is reached. While effective for geometry representation, this approach ignores the *computational cost* of interrogating the model, which is governed primarily by the complexity of the residual local expression rather than spatial bounds. To address this, a complexity-driven termination rule is proposed.

Complexity-driven termination Let $L(\mathcal{V})$ denote the *pruned complexity*, defined as the computational cost of the implicit function restricted to a voxel $\mathcal{V}$ after local pruning. The subdivision of $\mathcal{V}$ is terminated when:

$$L(\mathcal{V}) \leq L_t, \quad (1)$$

where L_t is a user-defined *complexity threshold*. This criterion ensures that subdivision continues only while it remains computationally advantageous: once the local expression is sufficiently simplified, further refinement is halted, avoiding unnecessary increases in memory usage and traversal depth.

This behavior is illustrated qualitatively in Fig. 1. Larger values of L_t arrest refinement earlier, reducing the leaf count but retaining more complex expressions in boundary voxels. Conversely, decreasing L_t forces finer subdivision, approaching the behavior of a conventional max-depth hierarchy as $L_t \rightarrow 0$. This formulation generalizes existing heuristics; for instance, targeting “one primitive per voxel” is equivalent to setting $L_t \approx 1$ under a primitive-count complexity model. As demonstrated in Section 6, optimal performance is typically found at intermediate thresholds, balancing hierarchy size against evaluation cost.

The metric $L(\mathcal{V})$ is determined by the specific implicit representation. For closed-form F-rep and SDFs, the number of arithmetic operations in the pruned expression serves as a robust proxy. For tree-structured representations (CSG trees, BlobTrees), $L(\mathcal{V})$ is approximated by the count of active primitives and operators remaining within $\mathcal{V}$. In the specific RPN-based implementation evaluated in this work (Section 4), $L(\mathcal{V})$ corresponds strictly to the number of active tokens (operators plus primitives) remaining in the voxel index list.

Traversal–evaluation trade-off The evaluation of an implicit function at a query point $\mathbf{x}$ involves two distinct costs: (i) traversing the spatial structure to locate the leaf voxel $\mathcal{V}_{\mathbf{x}}$ containing $\mathbf{x}$, and (ii) evaluating the pruned expression within that voxel. The total query time $T(\mathbf{x})$ is expressed as:

$$T(\mathbf{x}) = T_1(\mathbf{x}) + T_2(\mathbf{x}), \quad (2)$$

where T_1 is the traversal time and T_2 is the evaluation time.

For an adaptive hierarchy, the traversal cost T_1 grows with the depth of the leaf node $K(\mathcal{V}_x)$, while the evaluation cost T_2 depends on the local expression complexity $L(\mathcal{V}_x)$. A simplified linear cost model is therefore:

$$T(\mathbf{x}) \approx C_1 K(\mathcal{V}_x) + C_2 L(\mathcal{V}_x) + C_3, \quad (3)$$

where C_1 , C_2 , and C_3 are implementation-dependent constants representing traversal cost per level, evaluation cost per complexity unit, and fixed overheads, respectively.

A natural question arises: could a similar memory–performance trade-off be achieved simply by tuning the maximum depth $K_{\max}$ in conventional hierarchies? While increasing $K_{\max}$ invariably increases hierarchy occupancy, and therefore memory and traversal costs, it does not consistently improve end-to-end runtime. The reduction in per-voxel evaluation cost (T_2) gained from finer boundary refinement is frequently offset or dominated by the increased traversal overhead (T_1), resulting in a strictly scene-dependent optimum (Fig. 2).

This tuning challenge becomes even less transparent in production-grade sparse volume structures, such as N^3 -trees and VDB-style grids, where resolution and topology are governed by multiple interacting parameters Museth (2013); Uchytel and Storti (2023). In contrast, the proposed method introduces a single complexity parameter, L_t . Adjusting L_t generally forces changes in hierarchy depth levels to satisfy the complexity constraint. While this structural adaptation is often negligible for large thresholds, it becomes significant for small thresholds, where the hierarchy is forced to expand deeply, causing traversal costs to rise. This fundamental trade-off exists regardless of the specific spatial data structure; thus, although an octree is evaluated in this work, the threshold-based logic is equally applicable to other hierarchical grid structures.

Maximum unprunable complexity It is important to note that pruning cannot reduce local complexity arbitrarily at a fixed maximum resolution. For any given scene and maximum depth $K_{\max}$, there exists an upper bound on the remaining complexity across the finest voxels, defined as:

$$L_{\min} = \max_{\mathcal{V} \in \mathcal{H}_{\max}} L(\mathcal{V}), \quad (4)$$

where $\mathcal{H}_{\max}$ denotes the set of voxels at the finest available resolution. The variable $L_{\min}$ represents the *maximum unprunable complexity*, defined as the residual complexity that inevitably remains within a voxel when the chosen resolution is insufficient to fully resolve the local geometric or algebraic structure. For simple CSG models, $L_{\min}$ often approaches 1. However, for dense configurations with overlapping primitives (e.g., a cube defined by 6 intersecting half-spaces), $L_{\min}$ may remain significantly higher regardless of the grid depth. In practice, voxels reaching maximum complexity $L(\mathcal{V}) = L_{\min}$ typically occur only at the finest resolution and therefore constitute a small fraction of the spatial domain. As a result, their influence on the average performance of queries over spatially distributed points is minor, as demonstrated in Section 6.

Maximum residual complexity Consequently, the actual upper bound on the per-voxel complexity is determined by the greater of the user threshold and this irreducible limit:

$$\tilde{L} = \max(L_t, L_{\min}). \quad (5)$$

The variable $\tilde{L}$ represents the *maximum residual complexity* achievable under the specified constraints. Decreasing L_t below $L_{\min}$ cannot further reduce the complexity of the most difficult boundary regions; in these areas, it merely forces the hierarchy to expand fully to $K_{\max}$ without achieving the target simplicity. While this may still reduce the average complexity (L_{avg}) by refining simpler regions, the net performance benefit diminishes as traversal overhead begins to dominate.

Importantly, the proposed method does not require computing $L_{\min}$ a priori. In practice, it is sufficient to track the relationship between L_t and the currently observed complexity. When a voxel reaches the finest available resolution (maximum depth) while still satisfying $L(\mathcal{V}) > L_t$, its pruned complexity serves as

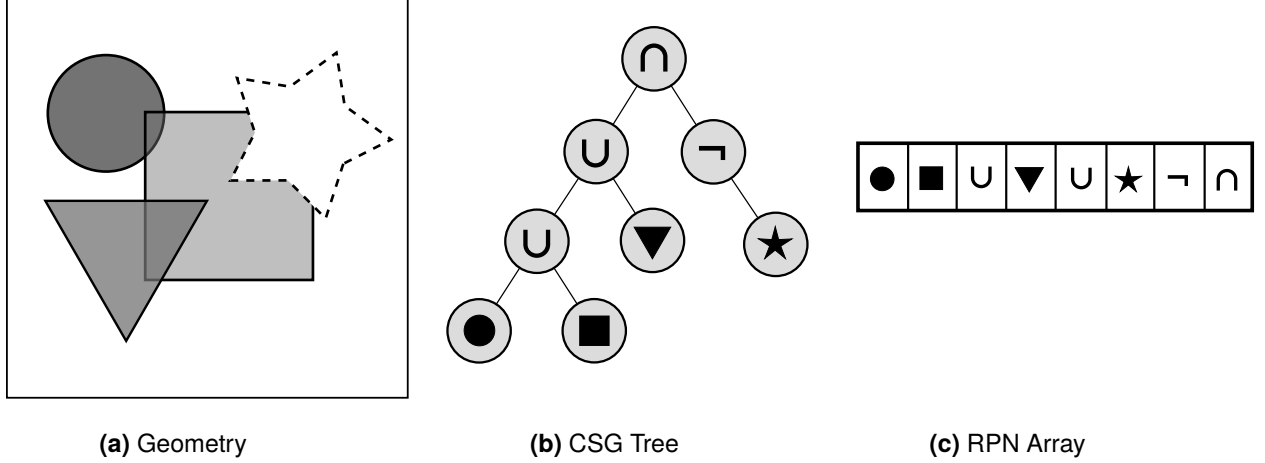


Figure 3. Representation of the implicit CSG model: (a) the resulting 2D geometry constructed from primitives, illustrating the subtractive effect of the star (dashed line), (b) the corresponding CSG tree, where leaves represent implicit primitives and internal nodes denote set-theoretic R-function operations, and (c) the linearized Reverse Polish Notation (RPN) array used as the contiguous memory layout for the pruning algorithm.

an observation of the local unprunable complexity. An estimate $\hat{L}_{\min}$ is maintained by updating $\hat{L}_{\min} \leftarrow \max(\hat{L}_{\min}, L(\mathcal{V}))$ whenever such a voxel is encountered. This estimate indicates whether the target threshold has been achieved during construction.

Finally, a conservative estimate of the *arithmetic speedup* can be defined as:

$$S_{\text{est}} = \frac{N}{\tilde{L}}, \quad (6)$$

where N is the complexity of the unpruned global expression. This metric represents the speedup for the *worst-case* voxel ($L(\mathcal{V}) = \tilde{L}$) and ignores traversal overhead (T_1). In practice, the realized average speedup is often considerably higher, since the worst-case complexity $\tilde{L}$ typically occurs only in a small region of the domain (at maximum resolution), while the majority of voxels reach the target threshold at much coarser levels. Furthermore, for large thresholds, where the hierarchy remains shallow and T_1 is negligible, S_{est} provides a reasonable lower bound for the total realizable speedup.

4 Pruning

This section presents an algorithm for pruning CSG trees composed of F-rep primitives and set-theoretic operations, realized via R-functions Rvachev (1975); Pasko et al. (1995), on a per-voxel basis. Existing pruning approaches typically simplify arithmetic structures, such as min/max compositions, using guaranteed range evaluation (e.g., interval arithmetic) Duff (1992); Snyder (1992); Keeter (2020) or rely on conservative variation bounds such as Lipschitz pruning Barbier et al. (2025). A key benefit of these approaches is that the pruned function preserves the sign and often the magnitude of the original field, thereby enabling subsequent processing tasks, such as discretization and polygonization, to be performed exactly as with the original implicit model.

For near-surface tasks such as ray-casting, however, a detailed function representation far from the surface is unnecessary. In an optimal rendering-oriented hierarchy, recursion is terminated for uniform regions (entirely inside or outside), and a constant label is stored; only boundary voxels retain a locally pruned expression. Consequently, a pruning procedure is integrated into the hierarchy construction, producing both voxel-local simplified expressions (Algorithm 1) and a voxel state (**IN**, **OUT**, or **BOUNDARY**).

Local expression representation The implicit models are stored globally in Reverse Polish Notation (RPN) as a contiguous token array $E = [e_1, \dots, e_N]$. For a given voxel $\mathcal{V}$, the locally pruned expression is represented by a sorted list of active (i.e., non-pruned) token indices $A(\mathcal{V}) = [a_1, \dots, a_L] \subseteq \{1, \dots, N\}$, where each retained token is addressed by its *global* index in the original RPN array. The resulting local expression is the subsequence of tokens $[e_{a_1}, \dots, e_{a_L}]$. An illustration of the implicit CSG model together with the CSG tree and the corresponding RPN representation is given in Fig. 3.

In the benchmarks, pruned expressions are typically sparse; consequently, storing only active indices yields a smaller memory footprint than a dense N -bit mask. During the pruning process, however, a *temporary* local bitset $\mathbf{m} \in \{0, 1\}^L$ is maintained over positions in A to support constant-time iteration and subtree deletion. Efficient traversal is provided by bit-scanning operations $\text{NEXTBIT}(\mathbf{m}, p)$ and $\text{PREVBIT}(\mathbf{m}, p)$, which locate active positions Leiserson and Prokop (1998). Initially, all bits are set to one ($m_i \leftarrow 1$). After pruning, the reduced index array is reconstructed by scanning the set bits in $\mathbf{m}$ and collecting the corresponding global indices, an operation denoted as $\text{COMPACT}(A, \mathbf{m})$.

Pruning algorithm For each active token e_i , a conservative interval enclosure $I_i(\mathcal{V}) = [\underline{f}_i, \overline{f}_i]$ is computed for the value of the corresponding subexpression over the domain $\mathbf{x} \in \mathcal{V}$ using interval arithmetic. The standard F-rep convention is assumed, where the solid interior corresponds to $F(\mathbf{x}) \geq 0$ and the exterior to $F(\mathbf{x}) < 0$. Consequently, the classification logic is defined as:

$$\begin{aligned} \text{Provably Inside on } \mathcal{V} &\iff \underline{f}_i \geq 0, \\ \text{Provably Outside on } \mathcal{V} &\iff \overline{f}_i < 0, \\ \text{Undetermined} &\iff 0 \in I_i. \end{aligned}$$

If an operand is provably inside or outside on $\mathcal{V}$, Boolean semantics allow the parent CSG node to be simplified. The pruning rules employed, summarized in Table 1, are expressed in terms of these guaranteed definitions and are independent of the specific R-function encoding.

The voxel-local pruning procedure, detailed in Algorithm 1, performs a single pass over the active RPN tokens, computing intervals and applying the rules from Table 1. When an operation simplifies to a single operand, the operator token is deactivated, and the discarded operand subtree is removed via dependency-clearing of the corresponding bits in the local mask (Algorithm 2). When an operation simplifies to a constant, the voxel state is immediately marked as **IN** or **OUT**. To support pruning of internal CSG nodes, a stack is maintained storing references to previously processed subtrees (parent indices and computed intervals).

Figure 4 illustrates the efficiency of this dependency-clearing mechanism. When a sub-expression evaluates to a dominating value (e.g., an **IN** operand in a Union), the sibling branches become mathematically irrelevant to the local result. Rather than recursively traversing these redundant subtrees, the algorithm identifies their linear span within the RPN array and explicitly clears the corresponding bits in the local mask $\mathbf{m}$. This operation effectively prunes the entire subtree from the active list in a single step, transforming the global expression into a sparse local representation eliminating additional arithmetic evaluation.

For boundary voxels, Algorithm 1 returns both the simplified index array A and the local complexity $L = |A|$ (the count of active RPN tokens). While L serves as the input metric for the termination criteria, the array A provides the optimized structure for runtime queries. During evaluation, the pruned CSG expression is reconstructed by iterating over the index array A and interpreting the corresponding token subsequence via a standard RPN stack.

Octree construction Algorithm 3 details the integration of voxel-local pruning into the octree construction process under the threshold-based stopping rule. For each voxel $\mathcal{V}$, pruning certifies the state (**IN/OUT**) or

Algorithm 1: Voxel-local pruning of a CSG tree in RPN form.

Input : RPN tokens E , voxel $\mathcal{V}$, active index list A_{init}
Output IN, OUT, or BOUNDARY(A, L)

:

1 **Function** PruneVoxel($E, \mathcal{V}, A_{\text{init}}$):

2 $A \leftarrow A_{\text{init}}$;

3 $\mathbf{m} \leftarrow \mathbf{1}$; ▷ local bitset over positions in A

4 $S \leftarrow \emptyset$; ▷ stores pairs (parentIndex, interval)

5 $p \leftarrow \text{NEXTBIT}(\mathbf{m}, 1)$;

6 **while** $p \leq |A|$ **do**

7 $i \leftarrow A[p]$;

8 $\text{token} \leftarrow e_i$;

9 **if** token is a primitive **then**

10 $I \leftarrow \text{INTERVALPRIMITIVE}(\text{token}, \mathcal{V})$;

11 $\text{PUSH}(S, (p, I))$;

12 **else if** token is NEGATION **then**

13 $(p_a, I_a) \leftarrow \text{POP}(S)$;

14 $I \leftarrow [-\overline{f}_a, -\underline{f}_a]$;

15 $\text{PUSH}(S, (p, I))$;

16 **else** ▷ token is UNION or INTERSECTION

17 $(p_b, I_b) \leftarrow \text{POP}(S)$;

18 $(p_a, I_a) \leftarrow \text{POP}(S)$;

19 **apply** Table 1 using I_a, I_b ;

20 **if** *simplifies to operand a* **then**

21 $\mathbf{m}_p \leftarrow 0$;

22 $\text{PruneSubtree}(E, A, \mathbf{m}, p_b, \text{nodes} = 1)$;

23 $\text{PUSH}(S, (p_a, I_a))$;

24 **else if** *simplifies to operand b* **then**

25 $\mathbf{m}_p \leftarrow 0$;

26 $\text{PruneSubtree}(E, A, \mathbf{m}, p_a, \text{nodes} = 1)$;

27 $\text{PUSH}(S, (p_b, I_b))$;

28 **else**

29 $I \leftarrow \text{INTERVALOP}(\text{token}, I_a, I_b)$;

30 $\text{PUSH}(S, (p, I))$;

31 $p \leftarrow \text{NEXTBIT}(\mathbf{m}, p + 1)$;

32 $(p_r, I_r) \leftarrow \text{POP}(S)$; ▷ root interval

33 **if** $0 \notin I_r$ **then**

34 **return** IN **if** $\underline{f}_r \geq 0$ **else** OUT;

35 **else**

36 $A \leftarrow \text{COMPACT}(A, \mathbf{m})$; ▷ collect active indices

37 $L \leftarrow |A|$; ▷ local complexity

38 **return** BOUNDARY(A, L);

Algorithm 2: RPN subtree pruning via dependency-clearing.

Input : RPN tokens E , active list A , bitset $\mathbf{m}$, position p_0 , $nodes$ **Output** Updated bitset $\mathbf{m}$

:

1 **Function** PruneSubtree($E, A, \mathbf{m}, p_0, nodes$):2 $count \leftarrow nodes$ 3 $p \leftarrow p_0$ 4 **while** $count > 0$ **do**5 $\mathbf{m}_p \leftarrow 0$; $count \leftarrow count - 1$ 6 $p \leftarrow \text{PREVBIT}(\mathbf{m}, p - 1)$ 7 **if** $count = 0$ **then**8 **break**9 $token \leftarrow E[A[p]]$ 10 **if** $token$ is **NEGATION** **then**11 $count \leftarrow count + 2$ 12 **if** $token$ is **UNION** or **INTERSECTION** **then**13 $count \leftarrow count + 3$

Table 1. Sign-based simplification rules. The notation $A \equiv \mathbf{IN}$ indicates $\underline{f}_A \geq 0$, while $A \equiv \mathbf{OUT}$ indicates $\overline{f}_A < 0$.

Operator	Condition	Simplification
$A \cup B$	$A \equiv \mathbf{OUT}$	B
	$B \equiv \mathbf{OUT}$	A
	$A \equiv \mathbf{IN} \vee B \equiv \mathbf{IN}$	$\mathbf{IN}$
$A \cap B$	$A \equiv \mathbf{IN}$	B
	$B \equiv \mathbf{IN}$	A
	$A \equiv \mathbf{OUT} \vee B \equiv \mathbf{OUT}$	$\mathbf{OUT}$
$\neg A$	$A \equiv \mathbf{IN}$	$\mathbf{OUT}$
	$A \equiv \mathbf{OUT}$	$\mathbf{IN}$

returns the reduced complexity $L(\mathcal{V})$. Boundary voxels (**BOUNDARY**) terminate when either the target complexity is reached ($L \leq L_t$) or the maximum depth $K_{\max}$ is encountered. As discussed in Section 3, if a voxel reaches $K_{\max}$ but still retains high complexity ($L > L_t$), its complexity is used to update the estimate $\hat{L}_{\min}$, providing a practical observation of the local unprunable complexity.

During octree construction, each child voxel initializes its pruning process using the already pruned active-index list A of the parent. Since Algorithm 1 processes only tokens referenced by the current list, this inherited active set significantly reduces per-voxel computational work and accelerates construction without altering the final pruning result.

5 Rendering

The implicit surface is visualized using standard first-hit ray-casting. Rays traverse the hierarchy in front-to-back order using a stack-based octree traversal algorithm. At each internal node, the visitation order of

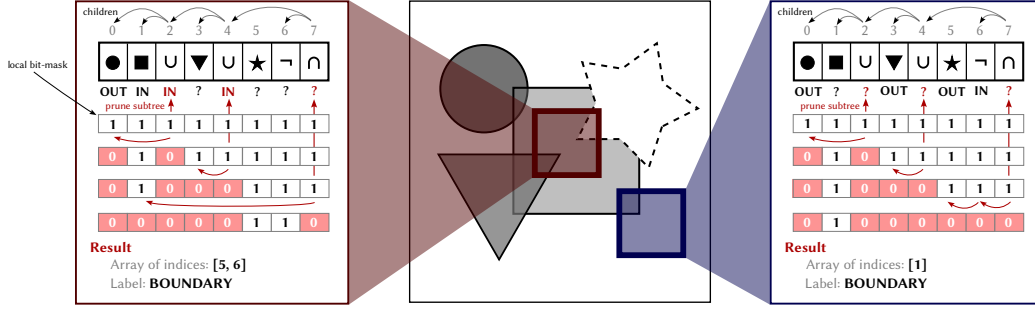


Figure 4. Visualization of the per-voxel pruning process (Algorithm 1) applied to the implicit model from Fig. 3. The pruning step begins with the global RPN array (Fig. 3(c)), where all tokens are initially active. **Red Voxel:** The Square primitive (index 1) is **IN**, which dominates the parent Unions (indices 2, 4), simplifying them to **IN**. This propagates to the root Intersection (index 7), reducing it to its right child (Negation), leaving only indices [5, 6]. **Blue Voxel:** The Circle (index 0) and Triangle (index 3) are **OUT**, removing them from the Unions. The Star (index 5) is **OUT**, simplifying the Negation (index 6) to **IN**. The root Intersection then simplifies to the remaining operand: the Square (index 1).

Algorithm 3: Complexity-driven octree construction.

Input : Root V_0 , RPN E , threshold L_t , max depth $K_{\max}$
Output Octree root R , online estimate $\hat{L}_{\min}$

```

1   $A_0 \leftarrow [1, \dots, N]$ 
2   $\hat{L}_{\min} \leftarrow 1$ 
3   $R \leftarrow \text{BuildNode}(V_0, 0, A_0)$ 
4  return  $(R, \hat{L}_{\min})$ 
5  Function  $\text{BuildNode}(V, k, A_{\text{init}})$ :
6       $(\text{type}, A, L) \leftarrow \text{PruneVoxel}(E, V, A_{\text{init}})$ 
7      if  $\text{type} \in \{\text{IN}, \text{OUT}\}$  then                                     ▶ Uniform voxel
8          return  $\text{LEAF}(\text{type})$ 
9      else                                                             ▶ Boundary voxel
10         if  $k = K_{\max} \vee L \leq L_t$  then
11             if  $k = K_{\max} \wedge L > L_t$  then
12                  $\hat{L}_{\min} \leftarrow \max(\hat{L}_{\min}, L)$ 
13             return  $\text{LEAF}(A)$ 
14         else
15              $u \leftarrow \text{INTERNALNODE}()$ 
16             foreach  $V_c$  in  $\text{SUBDIVIDE}(V)$  do
17                  $c \leftarrow \text{BUILDNODE}(V_c, k + 1, A)$ 
18                  $\text{ADDCHILD}(u, c)$ 
19             return  $u$ 

```

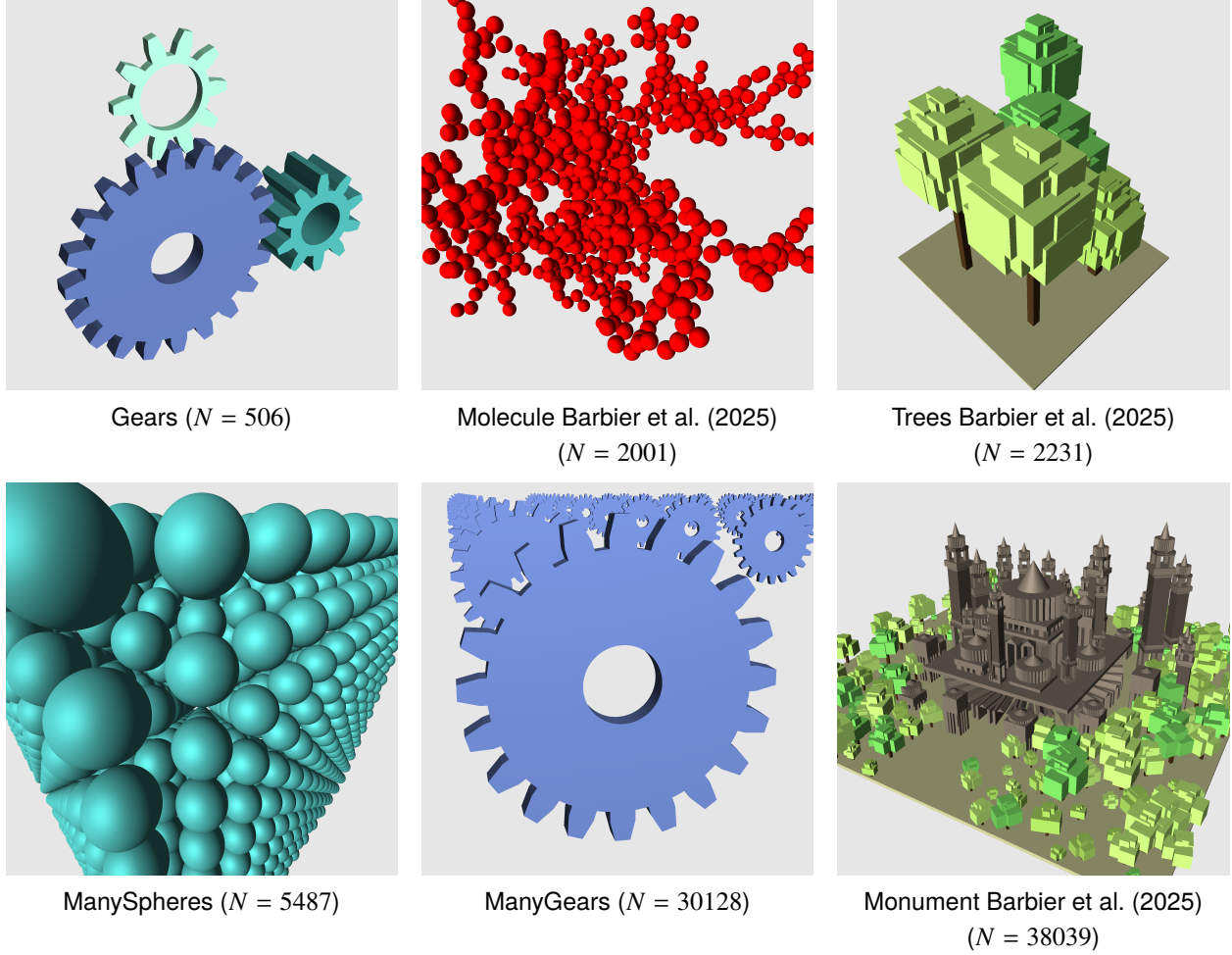


Figure 5. Benchmark scenes with original complexities N used throughout the evaluation.

child voxels is determined by a 3D differential analyzer (DDA) Amanatides and Woo (1987); child nodes are pushed to the stack in reverse intersection order to ensure correct front-to-back processing. Uniform voxels (**IN** and **OUT**) are skipped as they contain no zero-level set, restricting candidate intersections to boundary voxels. Within a boundary voxel, F is evaluated by iterating over the voxel-local index list and fetching the corresponding tokens from the global token array. The first surface intersection is located using *interval bisection* Fryazinov et al. (2010); Snyder (1992). Unlike standard scalar bisection which relies on bracketing a sign change, this method recursively subdivides the ray interval and rejects sub-segments where the function range strictly excludes zero ($0 \notin [f, \bar{f}]$). This ensures robust detection of surfaces even when the voxel entry and exit points share the same sign. Surface normals are computed from ∇F via central differences using the voxel-local pruned expression and shaded using the Phong illumination model Phong (1975).

6 Results

The proposed threshold-based stopping criterion and pruning pipeline are benchmarked on six CSG scenes, shown in Fig. 5: *ManySpheres*, *ManyGears*, *Gears*, *Trees*, *Molecule*, and *Monument*. The scenes *Trees*, *Molecule*, and *Monument* follow the sharp-Boolean setup used in prior work (e.g., Barbier et al., 2025);

the *Gears* scene is a CSG variant inspired by the gears example from Keeter (2020); and *ManySpheres* and *ManyGears* serve as stress tests with many repeated primitives to expose scaling trends. All timings are reported for a single-threaded CPU implementation and averaged over all evaluated points or pixels. Unless noted otherwise, speedups are measured relative to the unpruned baseline.

Section 6.1 analyzes the impact of varying the threshold L_t on pruning aggressiveness, hierarchy size, and runtime. The proposed method is then compared in Section 6.2 to Keeter’s interval pruning strategy Keeter (2020) and Lipschitz-based pruning Barbier et al. (2025) within the same evaluation framework.

6.1 Threshold-based approach

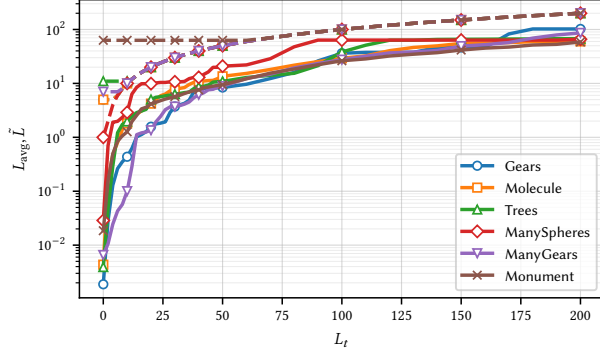
This section analyzes the trade-off between pruning aggressiveness, memory footprint, and runtime. Unless stated otherwise, the octree maximum depth is fixed at $K_{\max} = 10$. The special case $L_t = 0$ serves as the conventional baseline, where subdivision is driven solely by uniformity tests and the depth cap, preventing early termination of boundary voxels.

Pruned complexity versus threshold. Figure 6(a) illustrates the effect of the threshold on two key metrics: the maximum residual complexity $\tilde{L} = \max(L_t, L_{\min})$ and the measured spatial average over all leaves, L_{avg} . As predicted by the threshold stopping rule (Eq. (1)), decreasing L_t reduces the boundary complexity until it saturates at the maximum unprunable complexity $L_{\min}$, defined by Eq. (4), which is determined by the local structure of the model at the fixed maximum resolution. This floor varies considerably across scenes: *ManySpheres* approaches $L_{\min} \approx 1$, whereas *Monument* retains a much larger worst-case residual. Notably, L_{avg} remains consistently lower than $\tilde{L}$ because a large fraction of the spatial domain becomes uniform early, thereby eliminating the need for voxel-local expressions.

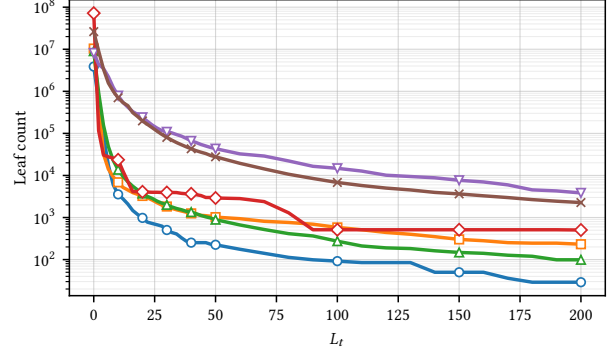
Octree occupancy and memory. Octree occupancy (Fig. 6(b)) is strongly influenced by both the scene geometry and the chosen threshold. The largest hierarchies occur at $L_t = 0$, where strict refinement is enforced even in regions where the pruned expression is already inexpensive. Increasing L_t reduces occupancy by multiple orders of magnitude while keeping $K_{\max}$ fixed. The steepest drop typically occurs near $L_t \approx 0$, reflecting that driving local complexity toward very small values requires high spatial resolution near the surface. For sufficiently large thresholds, occupancy often becomes approximately inversely proportional to L_t , consistent with the following idealized argument: if local complexity is roughly proportional to the number of primitives overlapping a cell, then allowing an eightfold increase in the complexity budget permits eight sibling leaves to be merged into their parent. This effectively decreases the leaf count eightfold in regions of uniformly distributed detail.

Table 2 quantifies the impact of this sparsity on the actual memory footprint. For complex scenes like *Monument*, the unpruned baseline consumes over 6.3 GB, which is prohibitive for many GPUs. The proposed method reduces memory to a manageable 2.1 GB at a conservative threshold of $L_t = 2$, and all the way down to just 16 MB at $L_t = 32$ with over 99% reduction. Even for dense stress tests like *ManyGears*, memory usage drops by nearly 50% with minimal pruning ($L_t = 2$). Efficiency gains appear most dramatic in the *ManySpheres* stress test, where memory usage collapses from 275 MB to just 440 KB at $L_t = 2$. This extreme drop highlights the ability of the method to adapt to scenes with high geometric density but low algebraic complexity, effectively replacing deep spatial subdivision with lightweight local index arrays.

Point evaluation speedup. Speedups for point evaluation are reported in Fig. 7, which separates the theoretical arithmetic gain from the realized total runtime. The arithmetic speedup (considering only the expression cost T_2), detailed in Fig. 7(b), increases monotonically as the threshold L_t decreases, reaching orders of magnitude for complex scenes. This confirms that the pruning algorithm effectively minimizes

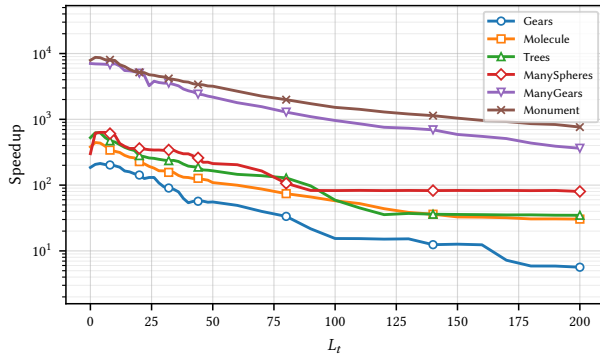


(a) Pruned complexity

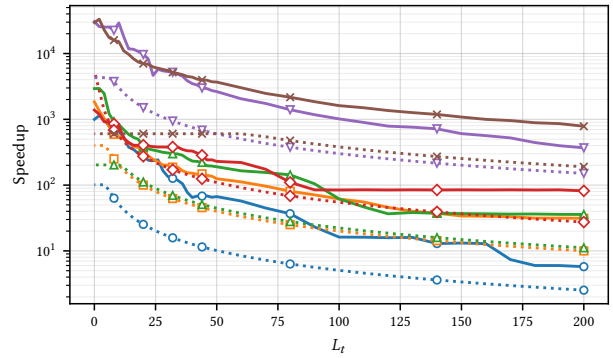


(b) Octree occupancy

Figure 6. Effect of the threshold L_t (active tokens) on hierarchy statistics. (a) Remaining local complexity after pruning. The maximum residual complexity $\tilde{L}$ (dashed) and spatial average L_{avg} (solid) are shown. (b) Octree occupancy (leaf count) as a function of threshold L_t (with fixed $K_{\text{max}} = 10$).



(a) Total speedup



(b) Arithmetic speedup

Figure 7. Decomposition of speedup into total and arithmetic components. (a) Realized total speedup showing the saturation caused by traversal overhead at low thresholds. (b) Measured arithmetic speedup (solid) compared to the conservative lower bound S_{est} (dotted, Eq. 6).

local evaluation complexity. However, the total speedup relative to the unpruned baseline, shown in Fig. 7(a), reveals the impact of hierarchy traversal. At low thresholds ($L_t \lesssim 25$), the traversal overhead (T_1) dominates, causing the end-to-end performance to stagnate or degrade despite the hyper-efficient local evaluation. This behavior is particularly evident in *ManySpheres*: while the arithmetic speedup continues to rise, the total speedup drops significantly at $L_t = 0$ due to excessive subdivision depth. Intermediate thresholds ($L_t \in [2, 10]$) generally achieve the best balance, maximizing the arithmetic benefit while limiting traversal costs.

Rendering time per pixel. The end-to-end rendering cost, as shown in Fig. 8, confirms these trends. Rendering time is minimized for small positive thresholds ($L_t \approx 2-5$). Setting $L_t = 0$ is slower because of excessive traversal overhead in regions that are already simple. Conversely, large L_t values slow rendering because rays spend more time evaluating complex expressions within coarse boundary voxels. This penalty is most pronounced in stress tests like *ManyGears*, where coarse voxels accumulate many primitives. Overall, moderate thresholds consistently provide the best balance, minimizing memory footprint and traversal overhead while maximizing local evaluation efficiency.

Table 2. Memory footprint (MB) at varying complexity thresholds L_t for fixed maximum depth $K_{\max} = 10$.

Scene	Complexity threshold (L_t)					
	0 (Base)	2	4	8	16	32
<i>Gears</i>	73.89	11.41	1.63	0.18	0.09	0.05
<i>Molecule</i>	197.90	10.95	0.61	0.25	0.22	0.20
<i>Trees</i>	378.38	39.61	6.52	0.95	0.32	0.21
<i>ManySpheres</i>	275.03	0.44	0.35	0.69	0.24	0.41
<i>ManyGears</i>	217.98	116.56	93.22	33.06	20.12	12.58
<i>Monument</i>	6313.40	2191.01	810.82	241.56	74.91	16.66

Table 3. Fitted constants of the linear runtime model (Eq. (3)). C_1 and C_2 are global fits across all scenes, while C_3 is reported as the observed range [min, max] across scenes.

Constant	Value	Units	Interpretation
C_1	8.97	ns	traversal cost per hierarchy level
C_2	22.88	ns	evaluation cost per active token
C_3	13 – 95	ns	scene-dependent constant overhead

Validation of the conservative bound. The experimental results validate the conservative nature of the arithmetic speedup bound S_{est} derived in Eq. (6). As shown in Fig. 7(b), the measured arithmetic speedup (solid curves) consistently exceeds the theoretical baseline (dotted curves)—often by an order of magnitude. This confirms that the pruning algorithm effectively exploits both uniform regions and pruned local expressions, where the residual complexity drops well below the worst-case limit $\tilde{L}$. However, Fig. 7(a) demonstrates that this massive arithmetic potential is not fully realized in the total speedup. While the arithmetic speedup in (b) continues to climb as $L_t \rightarrow 0$, the total speedup in (a) saturates or degrades. In this regime, the traversal overhead (T_1) becomes the dominant bottleneck. This trade-off is quantified in Table 3: the traversal cost per level ($C_1 \approx 9$ ns) is a significant fraction of the per-token evaluation cost ($C_2 \approx 23$ ns), meaning that deeper subdivision can quickly outweigh marginal pruning gains. Furthermore, the fixed overhead (C_3), which varies by scene (13–95 ns), imposes a hard floor on the minimum runtime, particularly in complex scenes where this overhead is substantial.

6.2 Comparison with other methods

The proposed method is compared against two state-of-the-art pruning baselines for accelerating evaluation: (i) Keeter’s interval pruning strategy Keeter (2020) and (ii) Lipschitz-based pruning Barbier et al. (2025). To ensure a fair comparison, both baselines were reimplemented in the same CPU framework and share identical octree construction, traversal, and evaluation backends. Crucially, to align the algorithmic objectives, the strict value-preserving constraints of the baselines were relaxed by enabling empty-space skipping for Keeter and far-field culling for Lipschitz. This effectively reduces them to sign-preserving methods that differ from the proposed method strictly in the efficiency of the local pruning logic (Section 4), while utilizing the same threshold-based termination rule (Section 3). The comparison focuses on three key dimensions: predictability, traversal efficiency, and memory footprint.

Predictability of local complexity. The purpose of the proposed threshold rule is not only to reduce cost, but also to make the *typical* per-leaf evaluation cost predictable: ideally, most boundary leaves should terminate with a local complexity close to the target threshold, up to the maximum unprunable complexity.

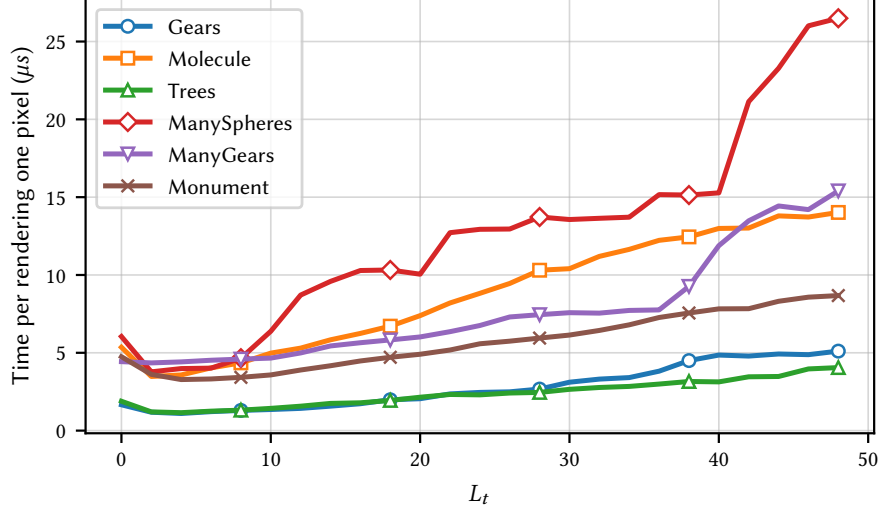


Figure 8. Average rendering time per pixel (μs) as a function of the threshold L_t (active tokens).

To quantify this effect, the average boundary-leaf complexity L_{avg} is compared with the maximum residual complexity $\tilde{L}$ (Eq. (5)). Figure 9 shows that the proposed pruning yields L_{avg} values that closely track $\tilde{L}$ across scenes and thresholds: when $L_t \geq L_{min}$, boundary leaves tend to stop near L_t , whereas for $L_t < L_{min}$, the average complexity saturates near the expected floor. In contrast, the baseline pruning approaches deviate more significantly from $\tilde{L}$, indicating less reliable convergence to the intended local complexity target and therefore less predictable average per-leaf evaluation cost.

Traversal efficiency and memory footprint. The end-to-end efficiency of the hierarchy is determined by its sparsity. Figure 10(a) compares the leaf count required by each method to achieve a given average pruned complexity L_{avg} . For the same L_{avg} (and therefore comparable per-sample evaluation cost), the proposed method typically requires significantly fewer leaves. This gap can reach orders of magnitude in complex scenes, reflecting that the proposed pruning enables earlier termination and more efficient local simplification in boundary regions. This sparsity yields two direct benefits: (1) a substantially reduced memory footprint, and (2) a decrease in the average leaf depth. Since traversal cost T_1 scales with depth, the proposed method achieves the same evaluation complexity with reduced traversal overhead compared to the baselines.

Construction efficiency. Finally, the cost of generating these optimized hierarchies is assessed. Figure 10(b) compares the total construction time required to achieve a given target complexity L_{avg} . While stricter pruning (lower L_{avg}) naturally increases build time due to deeper subdivision, the proposed method consistently incurs lower overhead than both baseline pruning approaches at any given level of simplification. This advantage stems from the *sign-preserving* nature of the proposed pruning (Algorithm 1). The baselines employ empty-space skipping (Keeter) and far-field culling (Lipschitz) to terminate subdivision in uniform regions, yet remain constrained by *value-preserving* logic: sub-expressions defining the field value must be retained even when they do not affect surface topology (e.g., retaining A in $\max(A, B)$ when both are negative). In contrast, all operations that do not influence the zero-level set are aggressively discarded in the proposed approach. Inherited token lists passed from parent to child voxels are thereby shortened, resulting in a cascading performance benefit that significantly reduces the total arithmetic work required during construction.

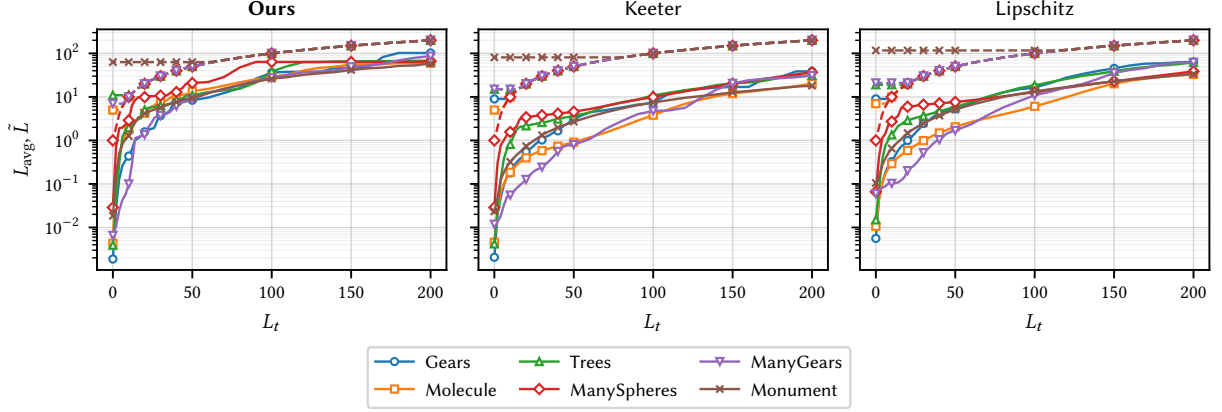


Figure 9. Predictability of *typical* local complexity under the threshold rule. The average pruned complexity L_{avg} (solid) is plotted against the target $\tilde{L} = \max(L_t, L_{\min})$ (dashed).

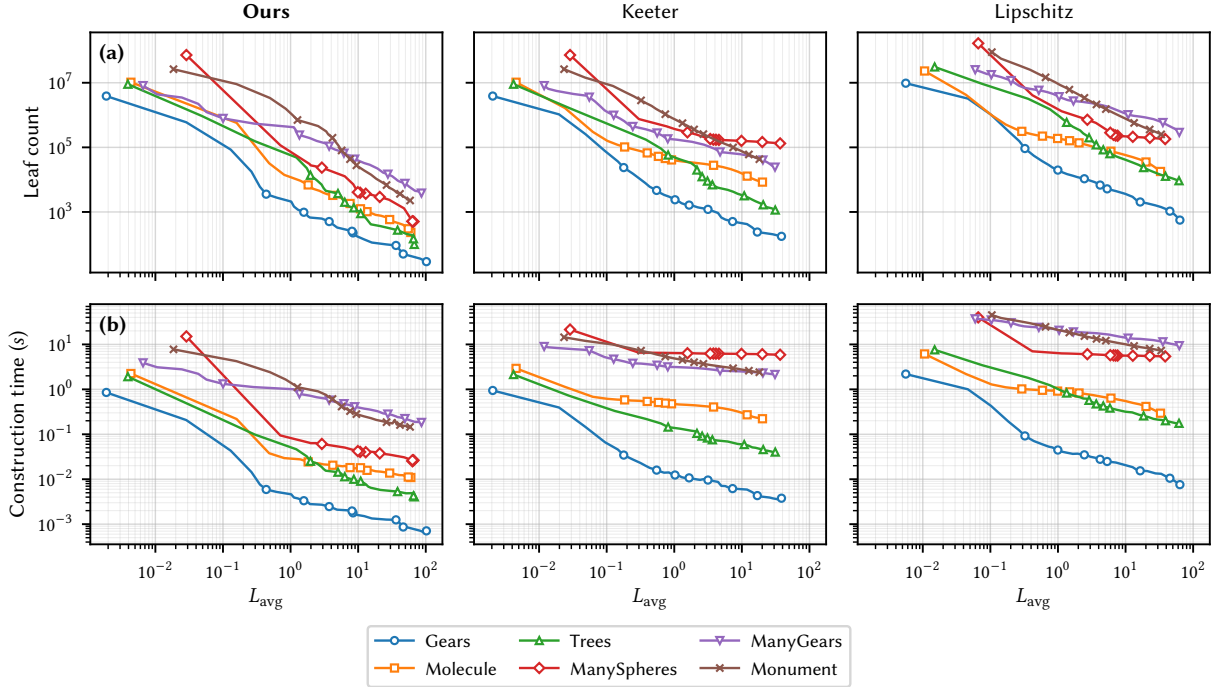


Figure 10. Comparison of hierarchy sparsity (a) and construction cost (b) plotted against average pruned complexity (L_{avg}). (a) The proposed method generates significantly sparser hierarchies than the baselines. (b) The proposed method incurs consistently lower build overhead than the baselines.

7 Conclusions, limitations and future work

This work presents a complexity-driven acceleration pipeline for implicit CSG that moves beyond arbitrary spatial depth limits via a threshold-based stopping criterion driven by local pruning. Shifting the construction paradigm from geometry-driven to complexity-driven, the proposed strategy explicitly balances the cost of hierarchy traversal against the computational complexity of the residual local expressions, enabling significant memory reductions while maintaining or improving rendering time compared to conventional max-depth approaches. Experimental results confirm that the proposed method yields sparser, more pre-

dictable hierarchies than existing interval- and Lipschitz-based baselines, effectively decoupling geometric resolution from evaluation cost in detailed constructive models. Ultimately, the proposed approach provides a flexible mechanism to optimize the performance-memory trade-off for high-fidelity implicit shape interrogation.

Limitations. The proposed method relies on the availability of conservative bounds for primitives and their transforms, since both pruning and uniformity classification are driven by guaranteed range evaluation. In scenes with fine geometric features, strong constructive interactions, or geometric singularities (e.g., sharp edges and corners), the maximum unprunable complexity ($L_{\min}$) can remain high, which limits the benefit of further reducing the threshold. Additionally, although the per-voxel index-array representation is memory-efficient compared to storing full local expressions or dense bit-masks, very large global token arrays may still benefit from additional compression or alternative layouts. Finally, the complexity metric $L(\mathcal{V})$ assumes uniform computational cost across all RPN tokens. Although specific operations vary in arithmetic intensity (e.g., trigonometric primitives vs. Boolean operators), the unweighted token count was found to be a sufficiently robust proxy for guiding subdivision without requiring architecture-specific tuning.

Future work. The presented framework opens several avenues for further development: (i) extending the pruning rules and range evaluation to support smooth blends and richer arithmetic F-rep beyond hard CSG, potentially incorporating tighter bounding techniques to reduce overestimation; (ii) developing GPU-oriented implementations and memory layouts for fast traversal to better exploit coherence and parallelism in ray casting and batched point queries; (iii) automating or adapting the selection of the complexity threshold through profiling or scene statistics to reduce the need for manual tuning across diverse models; and (iv) supporting incremental updates for interactive editing, where local regions of the hierarchy can be efficiently rebuilt or re-pruned after constructive changes.

References

- John Amanatides and Andrew Woo. A fast voxel traversal algorithm for ray tracing. In *Eurographics '87*, 1987. doi: 10.2312/EGTP.19871000.
- Wilhem Barbier, Mathieu Sanchez, Axel Paris, Élie Michel, Thibaud Lambert, Tamy Boubekur, Mathias Paulin, and Theo Thonat. Lipschitz pruning: Hierarchical simplification of primitive-based sdfs. *Computer Graphics Forum*, 44(2), 2025. doi: 10.1111/cgf.70057.
- Tom Duff. Interval arithmetic and recursive subdivision for implicit functions and constructive solid geometry. In *Proceedings of SIGGRAPH '92*, pages 131–138. ACM, 1992. doi: 10.1145/142920.134027.
- Eva Dyllong and Cornelius Grimm. A reliable extended octree representation of csg objects with an adaptive subdivision depth. In Roman Wyrzykowski, Jack Dongarra, Konrad Karczewski, and Jerzy Wasniewski, editors, *Parallel Processing and Applied Mathematics*, pages 1341–1350, Berlin, Heidelberg, 2008. Springer Berlin Heidelberg. ISBN 978-3-540-68111-3.
- Sarah F. Frisken, Ronald N. Perry, Alyn P. Rockwood, and Thouis R. Jones. Adaptively sampled distance fields: A general representation of shape for computer graphics. In *Proceedings of SIGGRAPH 2000*. ACM, 2000. doi: 10.1145/344779.344899.
- Oleg Fryazinov, Alexander Pasko, and Peter Comninos. Fast reliable interrogation of procedurally defined implicit surfaces using extended revised affine arithmetic. *Computers & Graphics*, pages 708–718, 2010. doi: 10.1016/j.cag.2010.07.003.

- John C. Hart. Sphere tracing: A geometric method for the antialiased ray tracing of implicit surfaces. *The Visual Computer*, 12(10):527–545, 1996. doi: 10.1007/s003710050084.
- Rama Karl Hoetzlein. Gvdb: Raytracing sparse voxel database structures on the gpu. In *High Performance Graphics 2016*, 2016. doi: 10.2312/hpg.20161197.
- Tao Ju, Frank Losasso, Scott Schaefer, and Joe Warren. Dual contouring of hermite data. In *Proceedings of SIGGRAPH 2002*, pages 339–346. ACM, 2002. doi: 10.1145/566570.566586.
- Matthew J. Keeter. Massively parallel rendering of complex closed-form implicit surfaces. *ACM Transactions on Graphics*, 39(4), 2020. doi: 10.1145/3386569.3392429.
- Aaron Knoll, Younis Hijazi, Andrew E. Kensler, Mathias Schott, Charles D. Hansen, and Hans Hagen. Fast ray tracing of arbitrary implicit surfaces with interval and affine arithmetic. *Computer Graphics Forum*, 28(1):26–40, 2009. doi: 10.1111/j.1467-8659.2008.01189.x.
- Samuli Laine and Tero Karras. Efficient sparse voxel octrees. In *Proceedings of I3D 2010*. ACM, 2010. doi: 10.1145/1730804.1730814.
- Charles E. Leiserson and Harald Prokop. Using de bruijn sequences to index a 1 in a computer word. Unpublished manuscript, June 1998.
- William E. Lorensen and Harvey E. Cline. Marching cubes: A high resolution 3D surface construction algorithm. In *Proceedings of SIGGRAPH '87*, pages 163–169. ACM, 1987. doi: 10.1145/37402.37422.
- Evgenii Maltsev, Dmitry Popov, Svyatoslav Chugunov, Alexander Pasko, and Iskander Akhatov. An accelerated slicing algorithm for frep models. *Applied Sciences*, 11(15), 2021. ISSN 2076-3417. doi: 10.3390/app11156767. URL <https://www.mdpi.com/2076-3417/11/15/6767>.
- Ken Museth. Vdb: High-resolution sparse volumes with dynamic topology. *ACM Transactions on Graphics*, 32(3), 2013. doi: 10.1145/2487228.2487235.
- Ken Museth. Nanovdb: A gpu-friendly and portable vdb data structure for real-time rendering and simulation. In *SIGGRAPH '21*. ACM, 2021. doi: 10.1145/3450623.3464653.
- Alexander Pasko, Valery Adzhiev, Alexander Sourin, and Vladimir Savchenko. Function representation in geometric modeling: Concepts, implementation and applications. *The Visual Computer*, 11:429–446, 1995. doi: 10.1007/BF02464333.
- Alexander Pasko, Oleg Fryazinov, Turlif Vilbrandt, Pierre-Alain Fayolle, and Valery Adzhiev. Procedural function-based modelling of volumetric microstructures. *Graph. Models*, 73(5):165–181, September 2011. ISSN 1524-0703. doi: 10.1016/j.gmod.2011.03.001. URL <https://doi.org/10.1016/j.gmod.2011.03.001>.
- G. Pasko, A. Pasko, M. Ikeda, and T. Kunii. Bounded blending operations. In *Proceedings SMI. Shape Modeling International 2002*, pages 95–103, 2002. doi: 10.1109/SMI.2002.1003533.
- Bui Tuong Phong. Illumination for computer generated pictures. *Communications of the ACM*, 18(6): 311–317, 1975.
- Dmitry Popov, Yulia Kuzminova, Evgenii Maltsev, Stanislav Evlashin, Alexander Safonov, Iskander Akhatov, and Alexander Pasko. Cad/cam system for additive manufacturing with a robust and efficient topology optimization algorithm based on the function representation. *Applied Sciences*, 11(16), 2021. ISSN 2076-3417. doi: 10.3390/app11167409. URL <https://www.mdpi.com/2076-3417/11/16/7409>.

- Aristides G. Requicha. Representations for rigid solids: Theory, methods, and systems. *ACM Computing Surveys*, 12(4):437–464, 1980. doi: 10.1145/356827.356833.
- Aldo Ricci. A constructive geometry for computer graphics. *The Computer Journal*, 16(2):157–160, 1973. doi: 10.1093/comjnl/16.2.157.
- Vladimir L. Rvachev. Method of r-functions in boundary-value problems. *Soviet Applied Mechanics*, 11(4): 345–354, 1975. doi: 10.1007/BF00882900.
- John M. Snyder. Interval analysis for computer graphics. In *Proceedings of SIGGRAPH '92*, pages 121–130. ACM, 1992. doi: 10.1145/142920.134024.
- Cole Uchytel and Duane Storti. A function-based approach to interactive high-precision volumetric design and fabrication. *ACM Transactions on Graphics*, 43(1), 2023. doi: 10.1145/3622934.
- Brian Wyvill, Andrew Guy, and Eric Galin. Extending the csg tree: Warping, blending and boolean operations in an implicit surface modeling system. *Computer Graphics Forum*, 18(2):149–158, 1999.
- Cédric Zanni. Synchronized tracing of primitive-based implicit volumes. *ACM Transactions on Graphics*, 44(1), 2024. doi: 10.1145/3702227.